

High-Level Synthesis with Simulated Annealing Scheduling

Oleg Kelner
Hochschule Hamm-Lippstadt
Lippstadt, Germany
oleg.kelner@stud.hshl.de

Abstract—High-level synthesis (HLS) scheduling, binding, and resource allocation are jointly NP-hard, and together determine the latency–area trade-off of the generated data path. This paper formulates the three sub-problems as a single simulated annealing (SA) search driven by one scalarized latency–area cost, with the number of allocated resource instances itself treated as a move within the search rather than fixed in advance. A C++ simulator implementing this formulation – seeded from an As-Soon-As-Possible (ASAP) schedule under a generous resource pool, with a shift/rebind/add-resource/remove-resource move set and a feasibility-restoring repair step – is evaluated on a 515-operation arithmetic data-flow graph. Across three cost-weighting configurations, the ASAP-seeded starting point is consistently a local minimum of the cost surface, and the search wanders away from it by a similar amount during exploration; the weighting instead determines what the low-temperature, near-greedy phase finds once it arrives – negligible further gain when latency dominates the cost, versus a sharp late collapse to a much lower cost when area dominates. This shows that the α/β weighting reshapes not just the final answer, but the shape of the search itself.

Index Terms—high-level synthesis, simulated annealing, scheduling, resource allocation, resource binding, metaheuristics

I. INTRODUCTION

High-level synthesis (HLS) compiles a behavioral description into a register-transfer-level data path by solving three interdependent sub-problems: *scheduling* assigns each operation of a data-flow graph (DFG) to a control step, *binding* maps each scheduled operation to a concrete functional-unit instance, and *allocation* decides how many instances of each resource type the data path actually contains. All three choices interact through the same resource-usage pattern over time, and jointly they are NP-hard, since the problem subsumes resource-constrained scheduling. Classical HLS tools sidestep this complexity with fast constructive heuristics – As-Soon-As-Possible (ASAP), As-Late-As-Possible (ALAP), and List Scheduling – that build a single schedule greedily, cycle by cycle. These methods are optimal only under restrictive assumptions (e.g. unlimited resources, or a fixed priority order that happens to match the true critical path) and, being purely constructive, cannot revise an earlier decision once made; they therefore get trapped in local optima of the latency–area cost surface.

Simulated annealing (SA) is a natural fit for escaping this trap: unlike greedy list scheduling, it accepts *worsening* moves with a temperature-controlled probability, which lets

the search leave local optima that a monotonically improving heuristic cannot escape, while still converging to a stable solution as the temperature is lowered. SA-based data-path synthesis also has direct precedent in VLSI CAD, where it was used to jointly allocate registers, functional units, and interconnect [1]. This places SA in contrast to exact formulations such as Integer Linear Programming (ILP), which can certify optimality but whose solve time grows too quickly with problem size to remain practical on realistic DFGs.

This paper makes the following contributions:

- a joint scheduling + binding + allocation SA formulation driven by a single weighted cost function (Section IV);
- a concrete SA move set and repair mechanism that keeps every candidate schedule feasible after each move (Section V);
- a worst-case complexity analysis of the scheduler’s per-move cost and total search cost across all runs (Section VI);
- an empirical study of how the α/β latency–area weighting reshapes the search landscape around the ASAP-seeded starting point, on a realistic DFG benchmark (Section VII).

The remainder of the paper first reviews related scheduling and allocation approaches (Section II), then formalizes the problem (Section III), before presenting the SA formulation, algorithm, complexity analysis, and experiments in turn.

II. RELATED WORK

Classical HLS tools schedule with fast constructive heuristics such as ASAP, ALAP, and List Scheduling, or with the more refined Force-Directed Scheduling, which balances concurrency across control steps to reduce resource cost under a fixed latency [2], [3]; exact formulations based on integer linear programming can certify optimal schedules but scale poorly with problem size [4]. Simulated annealing has previously been used to jointly allocate registers, functional units, and interconnect in data-path synthesis [1], and other metaheuristics such as genetic algorithms have likewise been applied to combined HLS scheduling and allocation [5]. SA is not specific to HLS: the same accept-worsening-moves-under-a-cooling-schedule mechanism underlies metaheuristic solvers for other combinatorial optimization problems, such as vehicle routing [6], which suggests the search dynamics studied here are a property of SA as an optimization method rather than

an artifact particular to data-path synthesis. This paper follows the HLS-specific SA-based line of work, with one difference: the number of allocated resource instances is itself a move within the same search, rather than being fixed in advance or optimized in a separate pass.

III. PROBLEM STATEMENT

Input. The input is a data-flow graph $G = (V, E)$, where each vertex $v \in V$ is a typed operation (e.g. LOAD, STORE, ADD, SUB, MUL, DIV), and each edge $(u, v) \in E$ denotes a data dependency requiring u to complete before v begins. A resource library specifies, for each resource type, an area cost, a fixed latency in clock cycles, and an available instance count.

Decision variables. A candidate solution assigns to every operation $v \in V$ a start cycle $c(v) \in \mathbb{N}$ and a resource-instance binding $r(v)$, identifying which physical instance of v 's resource type executes it. A solution is *feasible* iff it satisfies:

- 1) **Precedence:** for every edge $(u, v) \in E$, $c(v) \geq c(u) + \text{latency}(u)$;
- 2) **Resource exclusivity:** for any two operations $v_1 \neq v_2$ with $r(v_1) = r(v_2)$, their execution intervals $[c(v), c(v) + \text{latency}(v))$ do not overlap.

Objective. Informally, the goal is to find a feasible (c, r) , together with an instance count per resource type, that minimizes a weighted combination of total schedule latency and total allocated area; the precise weighted cost function is defined in Section IV.

Complexity. The joint problem is NP-hard, since fixing the resource counts and binding already reduces it to resource-constrained scheduling, itself a well-known NP-hard problem. Here allocation is not fixed but part of the search, so the combined search space – cycle assignment $\times$ binding $\times$ resource count – is strictly larger still. Even for DFGs of modest size, the number of feasible schedules grows far too quickly for exhaustive or exact search to remain practical, which motivates the simulated-annealing-based approach developed in Sections IV and V.

IV. MATHEMATICAL FOUNDATIONS

A. Cost Function

Every candidate solution (c, r, k) from Section III is scored by a single scalar cost,

$$\text{Cost}(c, r, k) = \alpha \cdot L(c) + \beta \cdot A(k), \quad (1)$$

combining two terms. The latency term $L(c) = \max_{v \in V} (c(v) + \text{latency}(v))$ is the schedule length in clock cycles, i.e. the completion time of the last operation. The area term $A(k) = \sum_t k_t \cdot \text{area}(t)$ is the total hardware cost of the allocated resource pool, summing the unit area of each resource type t over its allocated instance count k_t ; note that area depends only on how many instances are allocated, not on how many operations are bound to them. The weights $\alpha, \beta \geq 0$ let a designer trade latency against area: Equation (1) is a linear scalarization of the underlying

two-objective (latency, area) problem into the single objective that the search of Section V minimizes, and different choices of α/β correspond to different preferences on that trade-off.

B. Simulated Annealing Formalism

Simulated annealing explores the space of feasible solutions by repeatedly proposing a neighboring solution and deciding whether to move there. Let Cost_{cur} be the cost of the current solution and $\text{Cost}_{\text{next}}$ the cost of a proposed neighbor, and define $\Delta = \text{Cost}_{\text{next}} - \text{Cost}_{\text{cur}}$. The neighbor is accepted with the Metropolis probability

$$P(\Delta, T) = \begin{cases} 1, & \Delta \leq 0, \\ e^{-\Delta/T}, & \Delta > 0, \end{cases} \quad (2)$$

where T is the current *temperature*: improving moves are always accepted, while worsening moves are accepted with a probability that shrinks as Δ grows or T falls. Temperature is lowered geometrically,

$$T_{k+1} = \gamma \cdot T_k, \quad 0 < \gamma < 1, \quad (3)$$

starting from an initial temperature T_0 and stopping once T drops below a minimum threshold $T_{\min}$. At high temperature, Equation (2) accepts almost any move, so the search explores broadly and can escape local optima of the cost surface; as $T \rightarrow T_{\min}$, only improving or mildly worsening moves survive, so the search behaves increasingly like a greedy local search and settles into a minimum. A slow enough (e.g. logarithmic) cooling schedule is known to converge to a global optimum in the limit, but at a runtime cost that grows accordingly; the finite, geometric schedule of Equation (3) trades that asymptotic guarantee for a bounded number of iterations, which we return to in Section VI.

V. SIMULATED ANNEALING SCHEDULING ALGORITHM

A. State Representation and Initialization

A candidate solution is represented directly as the triple (c, r, k) from Section III: a start cycle $c(v)$ for every operation, a resource-instance binding $r(v)$, and a resource pool $k = (k_t)_t$ giving the number of allocated instances per resource type. The search is initialized with an ASAP schedule computed under the *full* resource pool declared in the configuration – every available instance of every type, not just the minimum needed. Each operation is placed at the earliest cycle its predecessors allow and greedily bound to a free instance, with the repair mechanism of Section V-B resolving any remaining conflicts. This initial solution is feasible by construction and, because the pool is generous, typically already close to latency-optimal; it is far from cost-optimal in another sense, however, since every declared instance counts toward its area whether the schedule actually uses it or not. Section VII-C returns to this starting point in detail, as it turns out to matter for how the search behaves.

B. Neighbor Generation and Repair

At each step, a neighboring solution is produced by applying one randomly chosen move to the current solution:

- 1) **Move forward**: shift a randomly chosen operation one cycle later.
- 2) **Move backward**: shift a randomly chosen operation one cycle earlier (clamped at cycle 0).
- 3) **Rebind**: reassign a randomly chosen operation to a different, compatible instance of its resource type.
- 4) **Add resource**: increase k_t by one for a chosen resource type t .
- 5) **Remove resource**: decrease k_t by one for a chosen resource type t (never below one), automatically rebinding any operations that were using the removed instance.

A move can, on its own, break one of the two feasibility conditions of Section III – for example moving an operation earlier than one of its predecessors, or binding two operations to the same instance at overlapping cycles. Every move is therefore followed by a *repair* step: *dependency repair* recursively pushes an operation’s successors forward whenever precedence is violated, and *resource-overlap repair* shifts operations forward whenever two operations sharing an instance now overlap in time. Because repair always restores a feasible solution, the Metropolis test of Equation (2) can be applied directly to the resulting cost, without any additional penalty term for infeasibility.

C. Acceptance, Cooling, and Multi-Run Strategy

Algorithm 1 summarizes the full outer loop. At each temperature level, a fixed number of neighbors are proposed, repaired, and accepted or rejected via Equation (2); the best solution found so far is tracked separately from the current solution, since a worsening move may still be accepted along the way. Once the proposals at a given temperature are exhausted, the temperature is cooled with Equation (3) and the process repeats until T falls below $T_{\min}$. Because the outcome of a single run depends on its random sequence of proposed moves, the scheduler performs several independent runs from different random seeds and reports the best solution found across all of them.

VI. RUNTIME AND OVERHEAD ANALYSIS

A. Theoretical Complexity

Generating a candidate neighbor is cheap: four of the five move types touch a single operation and run in $O(1)$, while rebinding is $O(m_t)$ in the number of available instances m_t of the relevant resource type, which is normally small. The repair step dominates instead: dependency repair can, in the worst case, propagate through every remaining successor of the moved operation, and resource-overlap repair can likewise touch every operation sharing that instance, so a single move-and-repair step costs $O(|V| + |E|)$ in the worst case. At a given temperature, I such steps are performed, and the number of temperature levels visited before T drops below $T_{\min}$ is

Algorithm 1 Simulated Annealing Scheduling (SAS)

Require: DFG $G = (V, E)$, resource library, T_0 , $T_{\min}$, γ , iterations per temperature level I

Ensure: best solution found S^*

```

1:  $S \leftarrow \text{ASAP}(G)$  with full resource pool
2:  $S^* \leftarrow S$ 
3:  $T \leftarrow T_0$ 
4: while  $T > T_{\min}$  do
5:   for  $i = 1$  to  $I$  do
6:      $S' \leftarrow \text{Repair}(\text{Move}(S))$ 
7:      $\Delta \leftarrow \text{Cost}(S') - \text{Cost}(S)$ 
8:     if  $\Delta \leq 0$  or  $\text{rand}(0, 1) < e^{-\Delta/T}$  then
9:        $S \leftarrow S'$ 
10:      if  $\text{Cost}(S) < \text{Cost}(S^*)$  then
11:         $S^* \leftarrow S$ 
12:      end if
13:    end if
14:  end for
15:   $T \leftarrow \gamma \cdot T$ 
16: end while
17: return  $S^*$ 
```

$\lceil \log_\gamma(T_{\min}/T_0) \rceil$. Repeating this for a number of independent runs R , the total worst-case work is

$$O\left(R \cdot \lceil \log_\gamma(T_{\min}/T_0) \rceil \cdot I \cdot (|V| + |E|)\right). \quad (4)$$

Every factor in Equation (4) except the graph size $|V| + |E|$ is a user-controlled parameter of the search, which makes the runtime-quality trade-off of Section IV-B explicit and tunable, at the cost of no longer offering an asymptotic optimality guarantee.

B. Overhead in Practice

The bound in Equation (4) is a worst case dominated by graph size and the number of accepted iterations; it is not a tight estimate of actual runtime, since a repair step rarely needs to touch the whole graph in practice – a moved operation’s precedence and resource conflicts are typically local to a small neighborhood of the DFG, not the whole graph, so the per-move cost seen in practice is usually far below the $O(|V| + |E|)$ bound it is measured against.

VII. SIMULATION

The concepts of Sections IV and V are implemented in a C++ simulator built around the model of Section III. This section walks through what the simulator consumes and produces, how its starting point is obtained, and then makes one observation about how the search behaves as the cost function’s weights change. Development of the C++ simulator was assisted by AI tools [7], [8].

A. Input: Data-Flow Graph and Configuration

The simulator takes two inputs. The DFG is a flat text file, one operation per line, giving an operation’s type and its operand identifiers; dependencies are inferred directly from

those operand references. The benchmark DFG used throughout this section follows this pattern:

```
t1  = LOAD(D1A1)
t9  = LOAD(D1B1)
t17 = MUL(t1, t9)
t18 = MUL(t2, t10)
t25 = ADD(t17, t18)
...
t511 = ADD(t510, t509)
t515 = DIV(t511, t514)
STORE(t515)
```

The full graph repeats this load–multiply–reduce pattern across several input vectors, combines the resulting partial sums through further addition and subtraction stages, and finishes with a division and a store, for 515 operations over the six operation types LOAD, STORE, ADD, SUB, MUL, and DIV.

Alongside the DFG, a JSON configuration fixes the SA parameters and the resource library:

```
{
  "sa": {
    "initial_temperature": 1,
    "cooling_rate": 0.95,
    "min_temperature": 0.005,
    "iterations_per_temp": 50,
    "runs": 10,
    "alpha": 0.75,
    "beta": 0.25,
    "seed": 6324798
  },
  "resources": {
    "LOAD": {"count": 25, "area": 5, "latency": 1},
    "STORE": {"count": 25, "area": 5, "latency": 1},
    "ADD": {"count": 25, "area": 5, "latency": 1},
    "SUB": {"count": 25, "area": 5, "latency": 1},
    "MUL": {"count": 25, "area": 15, "latency": 3},
    "DIV": {"count": 25, "area": 40, "latency": 4}
  }
}
```

`initial_temperature`, `min_temperature`, and `cooling_rate` are T_0 , $T_{\min}$, and γ from Equation (3); `iterations_per_temp` is I and `runs` is the number of independent restarts R , both from Algorithm 1; `alpha` and `beta` are the cost-function weights of Equation (1) – the two values varied in Section VII-D; and `seed` is a base value from which each of the R runs derives its own, distinct seed (an offset of the run index), so restarts are independent random streams while the whole experiment stays reproducible. Under `resources`, each entry instantiates one resource type from Section III: `count` is the maximum number of instances that may ever be allocated, `area` their unit area cost, and `latency` their fixed execution latency in cycles.

B. Output: The Resulting Schedule

The simulator’s output is a concrete assignment of every operation to a resource instance and a cycle interval – the (c, r, k) triple of Section V-A, made explicit. An excerpt of the best schedule found for the benchmark DFG: Each

TABLE I
EXCERPT OF THE RESULTING SCHEDULE (BEST SOLUTION FOUND).

Resource	Start	End	Operation
Res_7_LOAD	4	5	t1
Res_7_LOAD	0	1	t2
Res_3_MUL	6	9	t17
Res_1_ADD	9	10	t25

row fixes the start cycle $c(v)$, the binding $r(v)$ – a named instance, not just a resource type – and its cycle interval $[c(v), c(v) + \text{latency}(v))$. The set of distinct instance names appearing in the table, grouped by type, is the allocated pool k . Read cycle by cycle, this table is exactly the control-step schedule a downstream HLS back end would use to drive the datapath’s control logic.

C. From ASAP Baseline to Optimized Schedule

Simulated annealing does not start from an arbitrary or random solution: the initial state S_0 handed to Algorithm 1 is the ASAP-under-full-pool schedule described in Section V-A, instantiated here on the benchmark DFG. Because the declared pool is generous relative to the parallelism actually present in this DFG, S_0 is already close to latency-optimal – there is little room left to shorten the schedule further – while its area is essentially the configuration’s maximum, since every declared instance is allocated whether or not the schedule uses it, leaving a large part of the initial resource pool idle.

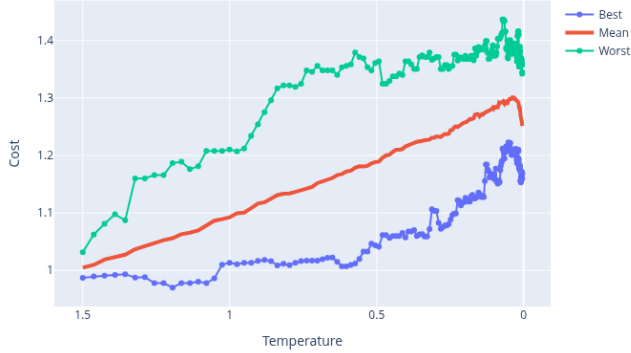
This asymmetry – an already-good latency paired with a deliberately wasteful area – is not an incidental implementation detail. As Section VII-D shows, it directly shapes how the search behaves once α and β change how costly that idle area is judged to be.

D. Cost-versus-Temperature Landscape

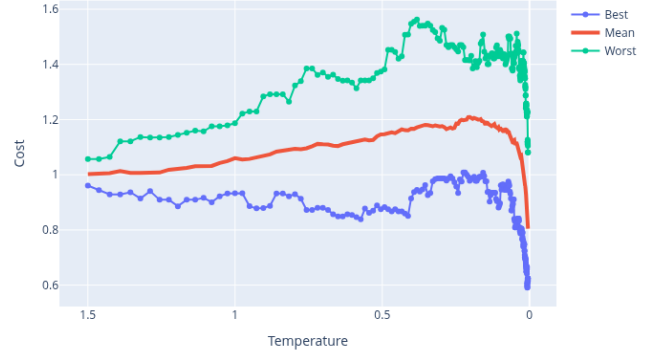
At each temperature level T , a run’s I proposals at that level end with the search in some particular current solution – exactly the solution that is carried forward, unchanged, into the next, cooler level. Rather than summarizing everything that happened during those I proposals, each run records only this single *resulting cost*: one number per run, per temperature level. Aggregating this resulting cost across all R independent runs at a given temperature then yields three quantities: the *best* (the minimum resulting cost across runs), the *worst* (the maximum resulting cost across runs), and the *mean* (the average resulting cost across runs). Every run also uses its own random seed (Section VII-A), so the spread between best and worst reflects genuine run-to-run variation and not a shared random sequence. Plotting these three quantities against temperature – with temperature decreasing along the axis, so the plot reads left to right from T_0 down to $T_{\min}$, start to end of the run – is the tool used below to visualize how the search behaves.

Figures 1–3 show this plot for three configurations that differ only in α and β : latency-optimizing ($\alpha = 0.75, \beta = 0.25$), balanced ($\alpha = 0.5, \beta = 0.5$), and area-optimizing ($\alpha = 0.25, \beta = 0.75$), all else held fixed.

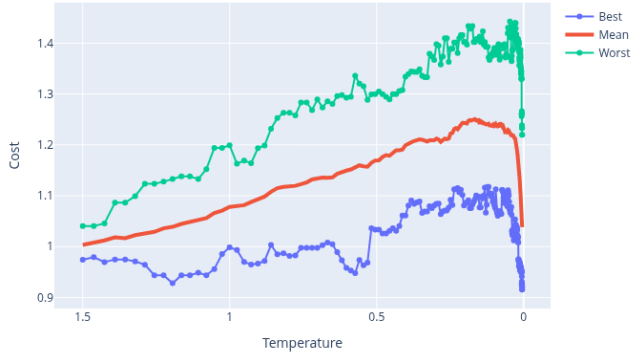
Cost vs Temperature (Resulting Solution per Run)

Fig. 1. Cost vs. temperature, latency-optimizing weighting ($\alpha = 0.75$, $\beta = 0.25$).

Cost vs Temperature (Resulting Solution per Run)

Fig. 3. Cost vs. temperature, area-optimizing weighting ($\alpha = 0.25$, $\beta = 0.75$).

Cost vs Temperature (Resulting Solution per Run)

Fig. 2. Cost vs. temperature, balanced weighting ($\alpha = 0.5$, $\beta = 0.5$).

The one point these three plots are meant to make is not about the mid-temperature exploration phase, which looks similar across all three weightings: in every configuration, S_0 is a local minimum of the cost surface, and the resulting cost climbs by a comparable amount as the search first moves away from it, since exploration at high temperature accepts worsening moves more or less irrespective of which term of Equation (1) is doing the charging. The difference shows up only in the low-temperature endgame. As T approaches $T_{\min}$, Equation (2) admits essentially only moves with $\Delta \leq 0$, so the search enters a near-greedy phase in which the only moves that survive are ones that lower cost – chiefly, shedding one of S_0 's many idle resource instances. Under the latency-optimizing weighting, that near-greedy phase has little to gain: removing an idle instance barely moves a cost dominated by the large α term, so the run ends close to, or even above, where it started – S_0 is, for practical purposes, the deepest point

the search reaches (Fig. 1). As β grows, the same shedding moves are worth far more, and the near-greedy phase has correspondingly more to gain: the final stretch of cooling turns from a negligible correction into a sharp collapse that ends well below the starting cost (Fig. 3), with the balanced weighting falling in between (Fig. 2).

Two caveats apply when reading these three plots side by side. First, the cost function itself changes between them, so the absolute cost values on the y -axis are not directly comparable across figures – only the *shape* of each curve, relative to its own starting cost, carries meaning. Second, that shape is not merely descriptive: it feeds back into the search through the Metropolis criterion of Equation (2), since the acceptance probability depends on Δ in the very units these plots show. Read this way, the three configurations do not differ in how far the search wanders during exploration, but in what the near-greedy endgame finds once it arrives: a latency-optimizing weighting leaves S_0 as effectively the deepest point reachable, while an area-optimizing weighting reveals a much deeper basin that the search only falls into in the last few temperature steps before $T_{\min}$.

VIII. CONCLUSION

Across all three α/β configurations, the ASAP-seeded starting point S_0 is a local minimum of the cost surface: during the mid-temperature exploration phase, the resulting cost per run rises away from S_0 by a comparable amount regardless of weighting. What differs is the low-temperature endgame, once the Metropolis criterion effectively admits only strictly improving moves. Under the latency-optimizing weighting, that near-greedy phase finds little to gain – shedding idle resource instances barely moves a cost dominated by the large α term – so the run ends at or above where it started, with S_0 effectively the best point the search reaches. As β grows, the same shedding moves pay off far more, and the final descent turns from negligible to dramatic: under the area-optimizing

weighting, the last stretch of cooling collapses the cost well below its starting value. The α/β weighting therefore does not change how far the search wanders during exploration – it changes whether a much deeper basin exists to fall into once the search turns greedy.

These results come with several limitations. The evaluation uses a single benchmark DFG, so generalization of this landscape shape to graphs with substantially different topology, resource mixes, or scale (e.g. CHStone or MachSuite kernels) remains untested. The comparison is internal to the SA search itself: no quantitative baseline against ASAP, ALAP, list scheduling, or force-directed scheduling is reported, so the absolute quality of the schedules found – not just their behavior under different weightings – is not yet established. The cooling schedule is a fixed geometric decay used for all three configurations; an adaptive or reheating schedule might behave differently on the very landscapes this paper characterizes. Finally, the SA formulation is compared against neither other metaheuristics (genetic algorithms, tabu search, particle swarm optimization) nor an exact ILP solution on a small-scale instance that could bound the optimality gap.

REFERENCES

- [1] S. Devadas and A. Newton, “Algorithms for hardware allocation in data path synthesis,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 8, pp. 768–781, 7 1989. [Online]. Available: <http://ieeexplore.ieee.org/document/31534/>
- [2] G. Micheli, “Synthesis and optimization of digital circuits,” *Choice Reviews Online*, vol. 32, pp. 32–0950–32–0950, 10 1994.
- [3] P. G. Paulin and J. P. Knight, “Force-directed scheduling for the behavioral synthesis of asic’s,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 8, pp. 661–679, 1989. [Online]. Available: <https://ieeexplore.ieee.org/document/31522>
- [4] C. T. Hwang, J. H. Lee, and Y. C. Hsu, “A formal approach to the scheduling problem in high level synthesis,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 10, pp. 464–475, 1991. [Online]. Available: <https://ieeexplore.ieee.org/document/75629>
- [5] M. Heijligers, L. Cluitmans, and J. Jess, “High-level synthesis scheduling and allocation using genetic algorithms,” *Proceedings of ASP-DAC’95/CHDL’95/VLSI’95 with EDA Technofair*, pp. 61–66, 1995. [Online]. Available: <https://ieeexplore.ieee.org/document/486203>
- [6] M. Blocho, “Heuristics, metaheuristics, and hyperheuristics for rich vehicle routing problems,” *Smart Delivery Systems: Solving Complex Vehicle Routing Problems*, pp. 101–156, 1 2020. [Online]. Available: <https://www.sciencedirect.com/science/chapter/edited-volume/abs/pii/B9780128157152000099>
- [7] Anthropic, “Claude [large language model],” 2026, used during software development, debugging, code generation and text drafting. Accessed: 2026-07-12. [Online]. Available: <https://claude.ai>
- [8] OpenAI, “ChatGPT (gpt-5.5) [large language model],” 2026, used during software development, debugging, code generation and text drafting. Accessed: 2026-07-12. [Online]. Available: <https://chatgpt.com>